PREP-AI TECHNICAL INTERVIEW SERIES

LeetCode Master Handbook

58 Curated FAANG & Blind-75 Solutions with Intuition & Big-O

WHAT THIS HANDBOOK COVERS

- [x] 15 Core DSA Topics: Arrays, Sliding Window, Trees, Graphs, DP...
- [x] Optimal, Production-Ready Python Solutions with Syntax Highlighting
- [x] Step-by-step Intuition & Algorithmic Thought Process
- [x] Strict Big-O Time Complexity & Space Complexity Breakdowns
- [x] Critical Edge Cases & Common FAANG Interview Pitfalls
- [x] Target Companies: Google, Amazon, Meta, Microsoft, Apple, Uber

Designed & Generated for PREP-AI Coding Arena Users

https://prepai.vercel.app - Pair Programming & Mock Interviews

Table of Contents - 58 Curated Problems

ARRAYS & HASHING (7 Problems)

#001 Two Sum [Easy]
#217 Contains Duplicate [Easy]
#242 Valid Anagram [Easy]
#049 Group Anagrams [Medium]
#347 Top K Frequent Elements [Medium]
#238 Product of Array Except Self [Medium]
#128 Longest Consecutive Sequence [Medium]

TWO POINTERS (5 Problems)

#125 Valid Palindrome [Easy]
#167 Two Sum II - Input Array Is Sorted [Medium]
#015 3Sum [Medium]
#011 Container With Most Water [Medium]
#042 Trapping Rain Water [Hard]

SLIDING WINDOW (4 Problems)

#121 Best Time to Buy and Sell Stock [Easy]
#003 Longest Substring Without Repeating Character
#424 Longest Repeating Character Replacement [Medi
#076 Minimum Window Substring [Hard]

STACK (4 Problems)

#020 Valid Parentheses [Easy]
#155 Min Stack [Medium]
#739 Daily Temperatures [Medium]
#084 Largest Rectangle in Histogram [Hard]

BINARY SEARCH (4 Problems)

#704 Binary Search [Easy]
#074 Search a 2D Matrix [Medium]
#033 Search in Rotated Sorted Array [Medium]
#004 Median of Two Sorted Arrays [Hard]

LINKED LIST (5 Problems)

#206 Reverse Linked List [Easy]
#021 Merge Two Sorted Lists [Easy]
#141 Linked List Cycle [Easy]
#019 Remove Nth Node From End of List [Medium]
#023 Merge k Sorted Lists [Hard]

TREES (5 Problems)

#226 Invert Binary Tree [Easy]
#104 Maximum Depth of Binary Tree [Easy]
#102 Binary Tree Level Order Traversal [Medium]
#098 Validate Binary Search Tree [Medium]
#124 Binary Tree Maximum Path Sum [Hard]

TRIES (1 Problems)

#208 Implement Trie (Prefix Tree) [Medium]

HEAP / PRIORITY QUEUE (2 Problems)

#215 Kth Largest Element in an Array [Medium]
#295 Find Median from Data Stream [Hard]

BACKTRACKING (3 Problems)

#078 Subsets [Medium]
#039 Combination Sum [Medium]
#051 N-Queens [Hard]

GRAPHS (4 Problems)

#200 Number of Islands [Medium]
#133 Clone Graph [Medium]
#207 Course Schedule [Medium]
#127 Word Ladder [Hard]

DYNAMIC PROGRAMMING (6 Problems)

#070 Climbing Stairs [Easy]
#198 House Robber [Medium]
#322 Coin Change [Medium]
#300 Longest Increasing Subsequence [Medium]

#1143 Longest Common Subsequence [Medium]

#072 Edit Distance [Medium]

GREEDY (2 Problems)

#053 Maximum Subarray [Medium]

#055 Jump Game [Medium]

INTERVALS (2 Problems)

#056 Merge Intervals [Medium]

#435 Non-overlapping Intervals [Medium]

BIT MANIPULATION (4 Problems)

#136 Single Number [Easy]

#191 Number of 1 Bits [Easy]

#338 Counting Bits [Easy]

#371 Sum of Two Integers [Medium]

#001. Two Sum

EASY

Category: Arrays & Hashing | Frequently Asked At: Google, Amazon, Microsoft, Meta

Problem Statement:

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to target. You may assume each input has exactly one solution, and you may not use the same element twice.

Constra

Optimal Approach: Hash Map (One-Pass)

Iterate through nums while storing each number and its index in a hash map. For each num, calculate complement = target - num. If complement exists in map, return [map[complement], current_index].

Time Complexity: O(N) - Single pass through the array with O(1) average lookup. **Space Complexity: O(N) - Hash map stores up to N elements.**

Edge Cases: Negative numbers, duplicates, solution at the first and last element.

Optimal Python Solution:

```
def twoSum(nums: list[int], target: int) -> list[int]:
    seen = {}
    for i, n in enumerate(nums):
        diff = target - n
        if diff in seen:
            return [seen[diff], i]
        seen[n] = i
    return []
```

#217. Contains Duplicate

EASY

Category: Arrays & Hashing | Frequently Asked At: Apple, Amazon, Microsoft, Adobe

Problem Statement:

Given an integer array `nums`, return `true` if any value appears at least twice in the array, and return `false` if every element is distinct.

Constra

Optimal Approach: Hash Set

Traverse the array and insert each element into a hash set. If an element is already in the set, a duplicate exists; return True. If loop finishes, return False.

Time Complexity: O(N) - Single pass through the array. **Space Complexity: O(N) - Set stores at most N elements.**

Edge Cases: Array with 1 element (always False), all identical elements, all distinct.

Optimal Python Solution:

```
def containsDuplicate(nums: list[int]) -> bool:
    seen = set()
    for n in nums:
        if n in seen:
            return True
        seen.add(n)
    return False
```

#242. Valid Anagram

EASY

Category: Arrays & Hashing | Frequently Asked At: Uber, Google, Amazon, Bloomberg

Problem Statement:

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise. An Anagram is a word or phrase formed by rearranging the letters of a different word or phrase.

Constra

Optimal Approach: Character Frequency Counting

If string lengths differ, they cannot be anagrams. Count character frequencies using a fixed-size 26-element array or counter. Verify both strings have identical character frequencies.

Time Complexity: O(N) - Where N is length of strings. **Space Complexity: O(1) - Fixed alphabet size (26 lowercase English letters).**

Edge Cases: Strings of unequal length, single character strings, identical strings.

Optimal Python Solution:

```
def isAnagram(s: str, t: str) -> bool:
    if len(s) != len(t):
        return False
    count = [0] * 26
    for c1, c2 in zip(s, t):
        count[ord(c1) - ord('a')] += 1
        count[ord(c2) - ord('a')] -= 1
    return all(c == 0 for c in count)
```

#049. Group Anagrams

MEDIUM

Category: Arrays & Hashing | Frequently Asked At: Amazon, Microsoft, Meta, Salesforce

Problem Statement:

Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Constra

Optimal Approach: Hash Map with Sorted Key / Frequency Tuple

Anagrams have identical sorted character sequences. Use the sorted string (or character count tuple) as a dictionary key and append matching strings to list values.

Time Complexity: O(N * K log K) - N strings of max length K sorted.Space Complexity: O(N * K) - Storing all strings in hash map values.

Edge Cases: Empty strings, single char strings, all words having no anagrams.

Optimal Python Solution:

```
from collections import defaultdict

def groupAnagrams(strs: list[str]) -> list[list[str]]:
    groups = defaultdict(list)
    for s in strs:
        key = ''.join(sorted(s))
        groups[key].append(s)
    return list(groups.values())
```

#347. Top K Frequent Elements

MEDIUM

Category: Arrays & Hashing | Frequently Asked At: Meta, Amazon, Google, ByteDance

Problem Statement:

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order. Your algorithm's time complexity must be better than O(n log n).

Constra

Optimal Approach: Min-Heap / Bucket Sort

Count frequencies with Counter. Use a bucket sort array where index is frequency (0 to N), and values are lists of numbers with that frequency. Collect top k from right to left.

Time Complexity: O(N) - Linear time using bucket sort. **Space Complexity: O(N) - Storing frequency count and bucket list.**

Edge Cases: k equals array length, all elements have equal frequency, negative numbers.

Optimal Python Solution:

```
from collections import Counter

def topKFrequent(nums: list[int], k: int) -> list[int]:
    count = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]
    for num, freq in count.items():
        buckets[freq].append(num)
    res = []
    for freq in range(len(buckets) - 1, 0, -1):
        for num in buckets[freq]:
            res.append(num)
            if len(res) == k:
                return res
    return res
```

#238. Product of Array Except Self

MEDIUM

Category: Arrays & Hashing | Frequently Asked At: Apple, Amazon, Microsoft, Goldman Sachs

Problem Statement:

Given an integer array `nums`, return an array `answer` such that `answer[i]` is equal to the product of all the elements of nums except nums[i]. You must write an algorithm that runs in O(n) time and without using the division operation.

Constra

Optimal Approach: Prefix and Suffix Products (Without Division)

Build output array using prefix product from left to right. Then do a second pass from right to left accumulating suffix product, multiplying it into the prefix value.

Time Complexity: O(N) - Two linear passes.

Space Complexity: O(1) - Output array does not count towards auxiliary space.

Edge Cases: Array contains one zero, multiple zeros, negative numbers.

Optimal Python Solution:

```
def productExceptSelf(nums: list[int]) -> list[int]:
    n = len(nums)
    res = [1] * n
    prefix = 1
    for i in range(n):
        res[i] = prefix
        prefix *= nums[i]
    postfix = 1
    for i in range(n - 1, -1, -1):
        res[i] *= postfix
        postfix *= nums[i]
    return res
```

#128. Longest Consecutive Sequence

MEDIUM

Category: Arrays & Hashing | Frequently Asked At: Google, Amazon, Spotify, Meta

Problem Statement:

Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence. You must write an algorithm that runs in O(n) time.

Constra

Optimal Approach: Hash Set & Sequence Start Detection

Insert all numbers into a set. A number is the start of a consecutive sequence only if (num - 1) is not in the set. For every sequence start, count forward (num + 1, num + 2...).

Time Complexity: O(N) - Each number visited at most twice. **Space Complexity: O(N) - Hash set for O(1) lookups.**

Edge Cases: Empty array, duplicate values, large numbers with integer gaps.

Optimal Python Solution:

```
def longestConsecutive(nums: list[int]) -> int:
    num_set = set(nums)
    longest = 0
    for n in num_set:
        if (n - 1) not in num_set:
            current_num = n
            streak = 1
            while (current_num + 1) in num_set:
                current_num += 1
                streak += 1
            longest = max(longest, streak)
    return longest
```

#125. Valid Palindrome

EASY

Category: Two Pointers | Frequently Asked At: Meta, Microsoft, Amazon, Apple

Problem Statement:

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.

Constra

Optimal Approach: Two Pointers (Left and Right)

Place pointers at start and end. Advance left and decrement right while skipping non-alphanumeric characters. Compare lowercased characters; return False on mismatch.

Time Complexity: O(N) - Single pass with two pointers. **Space Complexity: O(1) - Constant auxiliary space in-place.**

Edge Cases: Empty string or only spaces/punctuation (returns True), single character.

Optimal Python Solution:

```
def isPalindrome(s: str) -> bool:
    l, r = 0, len(s) - 1
    while l < r:
        while l < r and not s[l].isalnum():
            l += 1
        while l < r and not s[r].isalnum():
            r -= 1
        if s[l].lower() != s[r].lower():
            return False
        l += 1
        r -= 1
    return True
```

#167. Two Sum II - Input Array Is Sorted

MEDIUM

Category: Two Pointers | Frequently Asked At: Amazon, Google, Adobe

Problem Statement:

Given a 1-indexed array of integers `numbers` that is already sorted in non-decreasing order, find two numbers such that they add up to a specific `target` number. Return the indices of the two numbers, 1-indexed, as an integer array [index1, index2] of length 2.

Constra

Optimal Approach: Two Pointers on 1-Indexed Sorted Array

Because array is sorted: if numbers[l] + numbers[r] > target, decrease r. If sum < target, increase l. Return 1-indexed [l + 1, r + 1] when sum equals target.

Time Complexity: O(N) - Each element checked at most once.

Space Complexity: O(1) - In-place pointer traversal.

Edge Cases: Two elements only, negative target, large numbers.

Optimal Python Solution:

```
def twoSum(numbers: list[int], target: int) -> list[int]:
    l, r = 0, len(numbers) - 1
    while l < r:
        curr_sum = numbers[l] + numbers[r]
        if curr_sum == target:
            return [l + 1, r + 1]
        elif curr_sum > target:
            r -= 1
        else:
            l += 1
    return []
```

#015. 3Sum

MEDIUM

Category: Two Pointers | Frequently Asked At: Meta, Amazon, Apple, Product Startups

Problem Statement:

Given an integer array nums, return all the triplets `[nums[i], nums[j], nums[k]]` such that `i != j`, `i != k`, and `j != k`, and `nums[i] + nums[j] + nums[k] == 0`. Notice that the solution set must not contain duplicate triplets.

Constra

Optimal Approach: Sort Array + Two Pointers (Avoid Duplicates)

Sort nums. For each index i, if nums[i] == nums[i-1], skip to prevent duplicate triplets. Use two pointers on subarray (i+1 to end) to find pairs summing to -nums[i].

Time Complexity: O(N^2) - Sorting takes O(N log N) plus N * two-pointer passes. **Space Complexity: O(1) or O(N) - Dependent on sorting implementation.**

Edge Cases: All zeros ([0,0,0] -> [[0,0,0]]), duplicate numbers, no triplet possible.

Optimal Python Solution:

```
def threeSum(nums: list[int]) -> list[list[int]]:
    nums.sort()
    res = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        l, r = i + 1, len(nums) - 1
        while l < r:
            s = nums[i] + nums[l] + nums[r]
            if s == 0:
                res.append([nums[i], nums[l], nums[r]])
                while l < r and nums[l] == nums[l + 1]: l += 1
                while l < r and nums[r] == nums[r - 1]: r -= 1
                l += 1; r -= 1
            elif s < 0:
                l += 1
            else:
                r -= 1
    return res
```

#011. Container With Most Water

MEDIUM

Category: Two Pointers | Frequently Asked At: Google, Amazon, Goldman Sachs, Swiggy

Problem Statement:

You are given an integer array `height` of length `n`. There are `n` vertical lines drawn such that the two endpoints of the ith line are `(i, 0)` and `(i, height[i])`. Find two lines that together with the x-axis form a container, such that the container contains the most water. Return the maximum amount of water a container can store.

Constra

Optimal Approach: Two Pointers Greedy

Start pointers at both ends. Area is (r - l) * min(h[l], h[r]). The bottleneck is always the shorter line. Move the shorter line inward to potentially find a taller boundary.

Time Complexity: O(N) - Pointers meet after at most N steps. **Space Complexity: O(1) - Constant space.**

Edge Cases: Two bars only, all bars same height, strictly increasing/decreasing heights.

Optimal Python Solution:

```
def maxArea(height: list[int]) -> int:
    l, r = 0, len(height) - 1
    max_w = 0
    while l < r:
        area = (r - l) * min(height[l], height[r])
        max_w = max(max_w, area)
        if height[l] < height[r]:
            l += 1
        else:
            r -= 1
    return max_w
```

#042. Trapping Rain Water

HARD

Category: Two Pointers | Frequently Asked At: Google, Amazon, Microsoft, Meta

Problem Statement:

Given `n` non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Constra

Optimal Approach: Two Pointers with Left & Right Max Tracking

Maintain left_max and right_max. At any index, water trapped is determined by min(left_max, right_max) - height. Advance the pointer with smaller max height.

Time Complexity: O(N) - Single pass through heights. **Space Complexity: O(1) - Constant variables for tracking max heights.**

Edge Cases: Fewer than 3 elements (0 trapped), strictly descending/ascending terrain.

Optimal Python Solution:

```
def trap(height: list[int]) -> int:
    if not height: return 0
    l, r = 0, len(height) - 1
    l_max, r_max = height[l], height[r]
    water = 0
    while l < r:
        if l_max < r_max:
            l += 1
            l_max = max(l_max, height[l])
            water += l_max - height[l]
        else:
            r -= 1
            r_max = max(r_max, height[r])
            water += r_max - height[r]
    return water
```

#121. Best Time to Buy and Sell Stock

EASY

Category: Sliding Window | Frequently Asked At: Amazon, Microsoft, TCS Digital, Infosys

Problem Statement:

You are given an array `prices` where `prices[i]` is the price of a given stock on the ith day. You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock. Return the maximum profit you can achieve from this transaction.

Constra

Optimal Approach: One-Pass Running Minimum Price

Track the lowest stock price seen so far. At each day, calculate profit if sold today: price - min_price. Update max_profit accordingly.

Time Complexity: O(N) - Single pass.

Space Complexity: O(1) - Two variables min_price and max_profit.

Edge Cases: Prices strictly decrease (profit 0), price stays flat, 1 day only.

Optimal Python Solution:

```
def maxProfit(prices: list[int]) -> int:
    min_price = float('inf')
    max_profit = 0
    for p in prices:
        if p < min_price:
            min_price = p
        elif p - min_price > max_profit:
            max_profit = p - min_price
    return max_profit
```

#003. Longest Substring Without Repeating Characters

MEDIUM

Category: Sliding Window | Frequently Asked At: Amazon, Microsoft, Apple, Uber

Problem Statement:

Given a string `s`, find the length of the longest substring without repeating characters.

Constra

Optimal Approach: Sliding Window with Hash Map / Set

Keep a sliding window [l, r]. If character at r is already in window, increment l until character is removed. Window size is r - l + 1.

Time Complexity: O(N) - Each character added and removed at most once. **Space Complexity: O(min(N, M)) - Where M is alphabet size.**

Edge Cases: Empty string (0), string with all identical chars ('bbbbb' -> 1).

Optimal Python Solution:

```
def lengthOfLongestSubstring(s: str) -> int:
    seen = {}
    l = 0
    max_len = 0
    for r, char in enumerate(s):
        if char in seen and seen[char] >= l:
            l = seen[char] + 1
        seen[char] = r
        max_len = max(max_len, r - l + 1)
    return max_len
```

#424. Longest Repeating Character Replacement

MEDIUM

Category: Sliding Window | Frequently Asked At: Google, Amazon, Uber

Problem Statement:

You are given a string `s` and an integer `k`. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most `k` times. Return the length of the longest substring containing the same letter you can get after performing the above operations.

Constra

Optimal Approach: Sliding Window Tracking Max Frequency

Window is valid if (window_length - max_char_frequency) <= k. If invalid, increment left pointer by 1. The max window length ever reached is the answer.

Time Complexity: O(N) - Linear pass.

Space Complexity: O(1) - Max 26 uppercase alphabet counter.

Edge Cases: k exceeds string length, all characters already same, k = 0.

Optimal Python Solution:

```
from collections import defaultdict

def characterReplacement(s: str, k: int) -> int:
    count = defaultdict(int)
    max_f = 0
    l = 0
    for r in range(len(s)):
        count[s[r]] += 1
        max_f = max(max_f, count[s[r]])
        if (r - l + 1) - max_f > k:
            count[s[l]] -= 1
            l += 1
    return len(s) - l
```

#076. Minimum Window Substring

HARD

Category: Sliding Window | Frequently Asked At: Meta, Amazon, LinkedIn, Microsoft

Problem Statement:

Given two strings `s` and `t` of lengths `m` and `n` respectively, return the minimum window substring of `s` such that every character in `t` (including duplicates) is included in the window. If there is no such substring, return the empty string "".

Constra

Optimal Approach: Sliding Window with Target Frequency Counter

Expand right pointer until all characters from t are satisfied (formed == required). Then contract left pointer to minimize window while maintaining valid window condition.

Time Complexity: O(|S| + |T|) - Linear scan of both strings.

Space Complexity: O(|S| + |T|) - Frequency tables for window and target.

Edge Cases: len(s) < len(t), no valid substring exists, s == t.

Optimal Python Solution:

```
from collections import Counter

def minWindow(s: str, t: str) -> str:
    if not t or not s or len(s) < len(t): return ""
    target_cnt = Counter(t)
    required = len(target_cnt)
    window_cnt = {}
    l, formed = 0, 0
    ans = float("inf"), None, None
    for r, ch in enumerate(s):
        window_cnt[ch] = window_cnt.get(ch, 0) + 1
        if ch in target_cnt and window_cnt[ch] == target_cnt[ch]:
            formed += 1
        while l <= r and formed == required:
            if (r - l + 1) < ans[0]:
                ans = (r - l + 1, l, r)
            window_cnt[s[l]] -= 1
            if s[l] in target_cnt and window_cnt[s[l]] < target_cnt[s[l]]:
                formed -= 1
            l += 1
    return "" if ans[0] == float("inf") else s[ans[1]:ans[2] + 1]
```

#020. Valid Parentheses

EASY

Category: Stack | Frequently Asked At: Amazon, Microsoft, Meta, Bloomberg

Problem Statement:

Given a string `s` containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid. Open brackets must be closed by the same type of brackets and in the correct order.

Constra

Optimal Approach: Stack Matching

Push opening brackets onto stack. For closing brackets, check if stack is empty or top does not match corresponding opening bracket. Stack must be empty at the end.

Time Complexity: O(N) - Push and pop each character once.

Space Complexity: O(N) - Stack size in worst case (e.g. '((((((').

Edge Cases: Odd length string, closing bracket first, unclosed brackets.

Optimal Python Solution:

```
def isValid(s: str) -> bool:
    pairs = {')': '(', '}': '{', ']': '['}
    stack = []
    for c in s:
        if c in pairs:
            if not stack or stack[-1] != pairs[c]:
                return False
            stack.pop()
        else:
            stack.append(c)
    return len(stack) == 0
```

#155. Min Stack

MEDIUM

Category: Stack | Frequently Asked At: Amazon, Bloomberg, Google, Apple

Problem Statement:

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time. Implement the MinStack class with push(val), pop(), top(), and getMin().

Constra

Optimal Approach: Dual Stack (Value Stack + Min Stack)

Maintain main stack for values and auxiliary min_stack. When pushing x, push min(x, min_stack[-1]) to min_stack. getMin() simply returns min_stack[-1].

Time Complexity: O(1) - For all operations (push, pop, top, getMin). Space Complexity: O(N) - Auxiliary min stack.

Edge Cases: Multiple duplicate minimums, popping the minimum element.

Optimal Python Solution:

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        min_val = min(val, self.min_stack[-1] if self.min_stack else val)
        self.min_stack.append(min_val)

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]
```

#739. Daily Temperatures

MEDIUM

Category: Stack | Frequently Asked At: Meta, Amazon, Google

Problem Statement:

Given an array of integers `temperatures` represents the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the ith day to get a warmer temperature. If there is no future day for which this is possible, keep answer[i] == 0 instead.

Constra

Optimal Approach: Monotonic Decreasing Stack

Keep a stack of indices with decreasing temperatures. When current temperature > stack[-1], pop index and calculate days = current_index - prev_index.

Time Complexity: O(N) - Each index pushed and popped at most once. **Space Complexity: O(N) - Stack storing indices.**

Edge Cases: Temperatures strictly decrease (all zeros in result), all equal temps.

Optimal Python Solution:

```
def dailyTemperatures(temperatures: list[int]) -> list[int]:
    res = [0] * len(temperatures)
    stack = []  # indices
    for i, t in enumerate(temperatures):
        while stack and t > temperatures[stack[-1]]:
            prev = stack.pop()
            res[prev] = i - prev
        stack.append(i)
    return res
```

#084. Largest Rectangle in Histogram

HARD

Category: Stack | Frequently Asked At: Google, Amazon, Uber, DoorDash

Problem Statement:

Given an array of integers `heights` representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Constra

Optimal Approach: Monotonic Stack with Dummy Zero Boundaries

Maintain a stack of bar indices in ascending height order. When a shorter bar is encountered, pop previous bars; the popped bar is the shortest in the rectangle extending between current index and new stack top.

Time Complexity: O(N) - Each bar pushed and popped once.

Space Complexity: O(N) - Stack.

Edge Cases: Single bar, all bars same height, strictly ascending heights.

Optimal Python Solution:

```
def largestRectangleArea(heights: list[int]) -> int:
    stack = []
    max_area = 0
    heights.append(0)
    for i, h in enumerate(heights):
        while stack and heights[stack[-1]] > h:
            height = heights[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
        stack.append(i)
    heights.pop()
    return max_area
```

#704. Binary Search

EASY

Category: Binary Search | Frequently Asked At: Microsoft, Apple, TCS Digital, Wipro

Problem Statement:

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1. You must write an algorithm with O(log n) runtime complexity.

Constra

Optimal Approach: Standard Binary Search

Set l = 0, r = len(nums) - 1. Calculate mid = l + (r - l) // 2. If nums[mid] == target, return mid; if nums[mid] < target, l = mid + 1; else r = mid - 1.

Time Complexity: O(log N) - Search space halved every step. **Space Complexity: O(1) - Constant variables.**

Edge Cases: Target smaller than all elements, target larger, single element array.

Optimal Python Solution:

```
def search(nums: list[int], target: int) -> int:
    l, r = 0, len(nums) - 1
    while l <= r:
        mid = l + (r - l) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            l = mid + 1
        else:
            r = mid - 1
    return -1
```

#074. Search a 2D Matrix

MEDIUM

Category: Binary Search | Frequently Asked At: Amazon, Microsoft, Meta

Problem Statement:

You are given an `m x n` integer matrix `matrix` with two properties: integers in each row are sorted from left to right, and the first integer of each row is greater than the last integer of the previous row. Given an integer `target`, return true if target is in matrix or false otherwise. Must run in O(log(m * n)).

Constra

Optimal Approach: Treat 2D Matrix as 1D Virtual Array

Total elements = m * n. Binary search with indices 0 to m*n - 1. Convert index mid to matrix coordinates: row = mid // n, col = mid % n.

Time Complexity: O(log(m * n)) - Standard binary search on virtual 1D array. **Space Complexity: O(1) - In-place index calculations.**

Edge Cases: 1x1 matrix, target not in range, target is first/last element.

Optimal Python Solution:

```
def searchMatrix(matrix: list[list[int]], target: int) -> bool:
    if not matrix or not matrix[0]: return False
    m, n = len(matrix), len(matrix[0])
    l, r = 0, m * n - 1
    while l <= r:
        mid = l + (r - l) // 2
        val = matrix[mid // n][mid % n]
        if val == target:
            return True
        elif val < target:
            l = mid + 1
        else:
            r = mid - 1
    return False
```

#033. Search in Rotated Sorted Array

MEDIUM

Category: Binary Search | Frequently Asked At: Google, Amazon, Microsoft, Meta

Problem Statement:

There is an integer array `nums` sorted in ascending order (with distinct values). Prior to being passed to your function, nums is possibly rotated at an unknown pivot index. Given `nums` after possible rotation and an integer `target`, return the index of target if it is in nums, or -1 if it is not in nums. O(log n) runtime required.

Constra

Optimal Approach: Modified Binary Search (Check Sorted Half)

At least one half [l, mid] or [mid, r] is always sorted. Check if target lies within the sorted half; if yes, narrow search to that half, otherwise search the other half.

Time Complexity: O(log N) - Logarithmic search space reduction. **Space Complexity: O(1) - Constant space.**

Edge Cases: Array rotated at 0 (unrotated), target not present, 1 or 2 elements.

Optimal Python Solution:

```
def search(nums: list[int], target: int) -> int:
    l, r = 0, len(nums) - 1
    while l <= r:
        mid = l + (r - l) // 2
        if nums[mid] == target:
            return mid
        if nums[l] <= nums[mid]:
            if nums[l] <= target < nums[mid]:
                r = mid - 1
            else:
                l = mid + 1
        else:
            if nums[mid] < target <= nums[r]:
                l = mid + 1
            else:
                r = mid - 1
    return -1
```

#004. Median of Two Sorted Arrays

HARD

Category: Binary Search | Frequently Asked At: Google, Amazon, Microsoft, Goldman Sachs

Problem Statement:

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays. The overall run time complexity should be O(log (m+n)).

Constra

Optimal Approach: Binary Search on Partition of Smaller Array

Partition both arrays such that left halves contain half the elements and max(left) <= min(right). Binary search partition position in the smaller array.

Time Complexity: O(log(min(M, N))) - Binary search on smaller array. **Space Complexity: O(1) - Constant pointer variables.**

Edge Cases: One array empty, odd vs even total length, non-overlapping arrays.

Optimal Python Solution:

```
def findMedianSortedArrays(nums1: list[int], nums2: list[int]) -> float:
    A, B = nums1, nums2
    if len(B) < len(A):
        A, B = B, A
    total = len(A) + len(B)
    half = total // 2
    l, r = 0, len(A) - 1
    while True:
        i = (l + r) // 2
        j = half - i - 2
        Aleft = A[i] if i >= 0 else float("-inf")
        Aright = A[i + 1] if (i + 1) < len(A) else float("inf")
        Bleft = B[j] if j >= 0 else float("-inf")
        Bright = B[j + 1] if (j + 1) < len(B) else float("inf")
        if Aleft <= Bright and Bleft <= Aright:
            if total % 2:
                return min(Aright, Bright)
            return (max(Aleft, Bleft) + min(Aright, Bright)) / 2
        elif Aleft > Bright:
            r = i - 1
        else:
            l = i + 1
```

#206. Reverse Linked List

EASY

Category: Linked List | Frequently Asked At: Amazon, Microsoft, Apple, TCS Digital

Problem Statement:

Given the `head` of a singly linked list, reverse the list, and return the reversed list.

Constra

Optimal Approach: Iterative Three-Pointers (prev, curr, nxt)

Initialize prev = None. Traverse list with curr: save nxt = curr.next, reverse link curr.next = prev, move prev = curr and curr = nxt. Return prev.

Time Complexity: O(N) - Traverses each node once.

Space Complexity: O(1) - Constant pointer manipulation.

Edge Cases: Empty list (None), single node, two node list.

Optimal Python Solution:

```
def reverseList(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

#021. Merge Two Sorted Lists

EASY

Category: Linked List | Frequently Asked At: Amazon, Microsoft, Meta, Accenture

Problem Statement:

You are given the heads of two sorted linked lists `list1` and `list2`. Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists. Return the head of the merged linked list.

Constra

Optimal Approach: Dummy Head Pointer Merging

Create a dummy head node. Compare values of list1 and list2, appending smaller node to current pointer. Attach remaining non-empty list at the end.

Time Complexity: O(N + M) - Traverses both lists.

Space Complexity: O(1) - Splices existing nodes in place.

Edge Cases: One or both lists empty, lists of unequal lengths.

Optimal Python Solution:

```
def mergeTwoLists(list1, list2):
    dummy = ListNode()
    tail = dummy
    while list1 and list2:
        if list1.val < list2.val:
            tail.next = list1
            list1 = list1.next
        else:
            tail.next = list2
            list2 = list2.next
        tail = tail.next
    tail.next = list1 or list2
    return dummy.next
```

#141. Linked List Cycle

EASY

Category: Linked List | Frequently Asked At: Amazon, Microsoft, Goldman Sachs

Problem Statement:

Given `head`, the head of a linked list, determine if the linked list has a cycle in it. There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Return true if there is a cycle, otherwise false.

Constra

Optimal Approach: Floyd's Tortoise and Hare (Slow & Fast Pointers)

Move slow by 1 step and fast by 2 steps. If there is a cycle, fast will eventually lap and meet slow. If fast reaches None, no cycle exists.

Time Complexity: O(N) - Linear time. **Space Complexity: O(1) - Two pointer variables.**

Edge Cases: Empty list, single node without cycle, single node pointing to itself.

Optimal Python Solution:

```
def hasCycle(head) -> bool:
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

#019. Remove Nth Node From End of List

MEDIUM

Category: Linked List | Frequently Asked At: Google, Meta, Amazon, Apple

Problem Statement:

Given the `head` of a linked list, remove the `n`th node from the end of the list and return its head. Try doing it in one pass.

Constra

Optimal Approach: Two Pointers with Offset N

Advance fast pointer n + 1 steps ahead of slow (starting at dummy node). Then move both until fast reaches None. slow.next will be the node to remove.

Time Complexity: O(L) - Single pass where L is list length.

Space Complexity: O(1) - Dummy node and two pointers.

Edge Cases: Removing head node (n equals length), single node list.

Optimal Python Solution:

```
def removeNthFromEnd(head, n: int):
    dummy = ListNode(0, head)
    slow = fast = dummy
    for _ in range(n + 1):
        fast = fast.next
    while fast:
        slow = slow.next
        fast = fast.next
    slow.next = slow.next.next
    return dummy.next
```

#023. Merge k Sorted Lists

HARD

Category: Linked List | Frequently Asked At: Google, Amazon, Meta, Microsoft

Problem Statement:

You are given an array of `k` linked-lists `lists`, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Constra

Optimal Approach: Min-Heap / Priority Queue or Divide & Conquer

Push head of each list into a min-heap with (node.val, id(node), node). Pop the minimum, link it to merged list, and push its next node if present.

Time Complexity: O(N log k) - Where N is total nodes and k is number of linked lists **Space Complexity: O(k) - Heap stores at most k nodes.**

Edge Cases: Empty lists array, lists containing empty nodes [None, None].

Optimal Python Solution:

```
import heapq

def mergeKLists(lists):
    dummy = ListNode(0)
    curr = dummy
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
    while heap:
        val, i, node = heapq.heappop(heap)
        curr.next = node
        curr = curr.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

#226. Invert Binary Tree

EASY

Category: Trees | Frequently Asked At: Google, Amazon, Twitter, Capgemini

Problem Statement:

Given the `root` of a binary tree, invert the tree, and return its root.

Constra

Optimal Approach: Recursive Post-Order Swap

If root is None, return None. Swap root.left and root.right, then recursively invert both subtrees.

Time Complexity: O(N) - Visits each node once.

Space Complexity: O(H) - Recursion stack equal to tree height.

Edge Cases: Empty tree (None), single node, skewed tree.

Optimal Python Solution:

```
def invertTree(root):
    if not root:
        return None
    root.left, root.right = invertTree(root.right), invertTree(root.left)
    return root
```

#104. Maximum Depth of Binary Tree

EASY

Category: Trees | Frequently Asked At: Amazon, Microsoft, Apple, Wipro

Problem Statement:

Given the `root` of a binary tree, return its maximum depth. A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Constra

Optimal Approach: DFS Recursion (1 + max(left, right))

Base case: None node has depth 0. Recursive case: max depth is 1 + max(depth(root.left), depth(root.right)).

Time Complexity: O(N) - Visits all N nodes.

Space Complexity: O(H) - Recursion call stack height.

Edge Cases: Empty tree (0), single root node (1), degenerate tree.

Optimal Python Solution:

```
def maxDepth(root) -> int:
    if not root:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

#102. Binary Tree Level Order Traversal

MEDIUM

Category: Trees | Frequently Asked At: Amazon, Microsoft, Meta, LinkedIn

Problem Statement:

Given the `root` of a binary tree, return the level order traversal of its nodes' values (i.e., from left to right, level by level).

Constra

Optimal Approach: BFS with Level-by-Level Queue

Use a queue initialized with root. At each level, determine queue size, pop all nodes for that level, record their values, and enqueue their children.

Time Complexity: O(N) - Each node enqueued and dequeued once. Space Complexity: O(N) - Queue holds up to N/2 leaf nodes in worst case.

Edge Cases: Empty tree ([]), single node, unbalanced tree.

Optimal Python Solution:

```
from collections import deque

def levelOrder(root) -> list[list[int]]:
    if not root: return []
    res = []
    q = deque([root])
    while q:
        level = []
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left: q.append(node.left)
            if node.right: q.append(node.right)
        res.append(level)
    return res
```

#098. Validate Binary Search Tree

MEDIUM

Category: Trees | Frequently Asked At: Amazon, Microsoft, Meta, Bloomberg

Problem Statement:

Given the `root` of a binary tree, determine if it is a valid binary search tree (BST). A valid BST has left subtree keys strictly less than the node's key, and right subtree keys strictly greater.

Constra

Optimal Approach: DFS with Valid Range Propagation (-inf, +inf)

For a node to be a valid BST node, its value must satisfy low < node.val < high. For left child, high becomes node.val. For right child, low becomes node.val.

Time Complexity: O(N) - Each node visited once. **Space Complexity: O(H) - Call stack height.**

Edge Cases: Node value equals integer bounds, duplicate values (invalid BST).

Optimal Python Solution:

```
def isValidBST(root) -> bool:
    def validate(node, low=float('-inf'), high=float('inf')):
        if not node:
            return True
        if not (low < node.val < high):
            return False
        return (validate(node.left, low, node.val) and
                validate(node.right, node.val, high))
    return validate(root)
```

#124. Binary Tree Maximum Path Sum

HARD

Category: Trees | Frequently Asked At: Meta, Google, Amazon, ByteDance

Problem Statement:

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Given the root of a binary tree, return the maximum path sum of any non-empty path.

Constra

Optimal Approach: DFS Max Gain Computation

For each node, compute max non-negative branch gain: max(0, dfs(child)). Path passing through current node as highest point is node.val + left_gain + right_gain. Update global max.

Time Complexity: O(N) - Visits each node once.

Space Complexity: O(H) - Recursion stack.

Edge Cases: All negative node values, path doesn't include root.

Optimal Python Solution:

```
def maxPathSum(root) -> int:
    max_sum = float('-inf')
    def get_max_gain(node):
        nonlocal max_sum
        if not node: return 0
        left = max(get_max_gain(node.left), 0)
        right = max(get_max_gain(node.right), 0)
        max_sum = max(max_sum, node.val + left + right)
        return node.val + max(left, right)
    get_max_gain(root)
    return max_sum
```

#208. Implement Trie (Prefix Tree)

MEDIUM

Category: Tries | Frequently Asked At: Google, Amazon, Twitter

Problem Statement:

A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. Implement the Trie class with insert, search, and startsWith methods.

Constra

Optimal Approach: Trie (Prefix Tree) with Nested Hash Maps

Each TrieNode has children dictionary and is_end boolean flag. Traverse character by character: create missing children on insert, check existence on search/startsWith.

Time Complexity: O(L) - Where L is length of word for insert, search and startsWith. **Space Complexity: O(Total Characters) - Nodes created for unique prefix paths.**

Edge Cases: Empty string prefix, searching prefix that is not a word.

Optimal Python Solution:

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word: str) -> None:
        curr = self.root
        for c in word:
            if c not in curr.children:
                curr.children[c] = TrieNode()
            curr = curr.children[c]
        curr.is_end = True

    def search(self, word: str) -> bool:
        curr = self.root
        for c in word:
            if c not in curr.children: return False
            curr = curr.children[c]
        return curr.is_end

    def startsWith(self, prefix: str) -> bool:
        curr = self.root
        for c in prefix:
            if c not in curr.children: return False
            curr = curr.children[c]
        return True
```

#215. Kth Largest Element in an Array

MEDIUM

Category: Heap / Priority Queue | Frequently Asked At: Meta, Amazon, Microsoft, Google

Problem Statement:

Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array. Note that it is the kth largest element in the sorted order, not the kth distinct element. Can you solve it without sorting?

Constra

Optimal Approach: Min-Heap of Size K or QuickSelect

Maintain a min-heap of size k. For each element in nums, push to heap. If heap size exceeds k, pop the smallest element. After all elements, heap top is the kth largest.

Time Complexity: O(N log k) - Heap operations on at most k elements. **Space Complexity: O(k) - Stores k elements.**

Edge Cases: k = 1 (maximum element), k = N (minimum element), duplicates.

Optimal Python Solution:

```
import heapq

def findKthLargest(nums: list[int], k: int) -> int:
    heap = []
    for n in nums:
        heapq.heappush(heap, n)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]
```

#295. Find Median from Data Stream

HARD

Category: Heap / Priority Queue | Frequently Asked At: Google, Amazon, Apple, Goldman Sachs

Problem Statement:

The median is the middle value in an ordered integer list. Implement the MedianFinder class with two heaps: addNum(num) and findMedian().

Constra

Optimal Approach: Two Heaps (Max-Heap for Lower Half, Min-Heap for Upper Half)

Store smaller half in max_heap and larger half in min_heap. Balance heaps so sizes differ by at most 1. Median is top of larger heap or average of both tops.

Time Complexity: O(log N) - For addNum; O(1) for findMedian. **Space Complexity: O(N) - Storing all numbers across two heaps.**

Edge Cases: Stream with duplicate values, even vs odd count of elements.

Optimal Python Solution:

```
import heapq

class MedianFinder:
    def __init__(self):
        self.small = []  # max-heap (invert values)
        self.large = []  # min-heap

    def addNum(self, num: int) -> None:
        heapq.heappush(self.small, -num)
        if self.small and self.large and (-self.small[0] > self.large[0]):
            heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.small) > len(self.large) + 1:
            heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small) + 1:
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self) -> float:
        if len(self.small) > len(self.large):
            return -self.small[0]
        if len(self.large) > len(self.small):
            return self.large[0]
        return (-self.small[0] + self.large[0]) / 2.0
```

#078. Subsets

MEDIUM

Category: Backtracking | Frequently Asked At: Amazon, Meta, Bloomberg, Uber

Problem Statement:

Given an integer array `nums` of unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets. Return the solution in any order.

Constra

Optimal Approach: Backtracking / Cascading

At each index i, we have two choices: include nums[i] in subset or exclude it. Explore both paths recursively, adding current subset copy to results.

Time Complexity: O(N * 2^N) - 2^N subsets generated, copying takes O(N). **Space Complexity: O(N) - Recursion tree depth.**

Edge Cases: Empty array (returns [[]]), single element.

Optimal Python Solution:

```
def subsets(nums: list[int]) -> list[list[int]]:
    res = []
    subset = []
    def backtrack(i):
        if i >= len(nums):
            res.append(subset.copy())
            return
        subset.append(nums[i])
        backtrack(i + 1)
        subset.pop()
        backtrack(i + 1)
    backtrack(0)
    return res
```

#039. Combination Sum

MEDIUM

Category: Backtracking | Frequently Asked At: Airbnb, Amazon, Meta, Uber

Problem Statement:

Given an array of distinct integers `candidates` and a `target` integer, return a list of all unique combinations of candidates where the chosen numbers sum to target. The same number may be chosen unlimited number of times.

Constra

Optimal Approach: Backtracking with Reuse (Branch Pruning)

Try adding candidates[i]. If total sum == target, record combination. If sum > target, prune branch. Call backtrack(i) to allow reusing same element.

Time Complexity: O(2^(T/M)) - Where T is target and M is min candidate value. **Space Complexity: O(T/M) - Maximum recursion stack depth.**

Edge Cases: No combination possible, target equals a candidate value.

Optimal Python Solution:

```
def combinationSum(candidates: list[int], target: int) -> list[list[int]]:
    res = []
    def dfs(i, cur, total):
        if total == target:
            res.append(cur.copy())
            return
        if i >= len(candidates) or total > target:
            return
        cur.append(candidates[i])
        dfs(i, cur, total + candidates[i])
        cur.pop()
        dfs(i + 1, cur, total)
    dfs(0, [], 0)
    return res
```

#051. N-Queens

HARD

Category: Backtracking | Frequently Asked At: Google, Amazon, Microsoft, Goldman Sachs

Problem Statement:

The n-queens puzzle is the problem of placing `n` queens on an `n x n` chessboard such that no two queens attack each other. Given an integer `n`, return all distinct solutions to the n-queens puzzle.

Constra

Optimal Approach: Backtracking with Column & Diagonal Sets

Place queen row by row. Track occupied columns, positive diagonals (r + c), and negative diagonals (r - c) in sets to achieve O(1) conflict checking.

Time Complexity: O(N!) - Only valid queen placements explored per row. **Space Complexity: O(N) - Storage for board state and conflict sets.**

Edge Cases: N = 1 (valid), N = 2 and N = 3 (no solution exists).

Optimal Python Solution:

```
def solveNQueens(n: int) -> list[list[str]]:
    col, pos_diag, neg_diag = set(), set(), set()
    res = []
    board = [["."] * n for _ in range(n)]
    def backtrack(r):
        if r == n:
            res.append(["".join(row) for row in board])
            return
        for c in range(n):
            if c in col or (r + c) in pos_diag or (r - c) in neg_diag:
                continue
            col.add(c); pos_diag.add(r + c); neg_diag.add(r - c)
            board[r][c] = "Q"
            backtrack(r + 1)
            col.remove(c); pos_diag.remove(r + c); neg_diag.remove(r - c)
            board[r][c] = "."
    backtrack(0)
    return res
```

#200. Number of Islands

MEDIUM

Category: Graphs | Frequently Asked At: Amazon, Microsoft, Google, Bloomberg

Problem Statement:

Given an `m x n` 2D binary grid `grid` which represents a map of '1's (land) and '0's (water), return the number of islands. An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically.

Constra

Optimal Approach: BFS / DFS Flood Fill

Iterate through each cell. When an unvisited land cell '1' is found, increment island count and perform BFS/DFS to sink all connected land cells by setting them to '0'.

Time Complexity: O(M * N) - Each cell visited at most a constant number of times. **Space Complexity: O(M * N) - Call stack / queue for full grid island.**

Edge Cases: Grid of all water ('0'), grid of all land ('1'), 1x1 grid.

Optimal Python Solution:

```
def numIslands(grid: list[list[str]]) -> int:
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    def dfs(r, c):
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] != "1":
            return
        grid[r][c] = "0"
        dfs(r + 1, c); dfs(r - 1, c); dfs(r, c + 1); dfs(r, c - 1)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "1":
                count += 1
                dfs(r, c)
    return count
```

#133. Clone Graph

MEDIUM

Category: Graphs | Frequently Asked At: Meta, Amazon, Microsoft

Problem Statement:

Given a reference of a node in a connected undirected graph. Return a deep copy (clone) of the graph. Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

Constra

Optimal Approach: Hash Map with DFS / BFS Graph Traversal

Use a hash map to map original nodes to their cloned counterparts (old_to_new). If a node is already in the map, return the clone. Otherwise clone node and clone its neighbors recursively.

Time Complexity: O(V + E) - Visits every vertex and edge once. **Space Complexity: O(V) - Hash map storing all V cloned nodes.**

Edge Cases: Empty graph (None), graph with 1 node and no neighbors, cyclical graph.

Optimal Python Solution:

```
def cloneGraph(node):
    if not node: return None
    clones = {}
    def dfs(curr):
        if curr in clones:
            return clones[curr]
        copy = Node(curr.val)
        clones[curr] = copy
        for n in curr.neighbors:
            copy.neighbors.append(dfs(n))
        return copy
    return dfs(node)
```

#207. Course Schedule

MEDIUM

Category: Graphs | Frequently Asked At: Amazon, Google, Microsoft, Meta

Problem Statement:

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [a, b]` indicates that you must take course `b` first if you want to take course `a`. Return true if you can finish all courses. Otherwise, return false.

Constra

Optimal Approach: Topological Sort (Kahn's In-Degree / DFS Cycle Detection)

Build adjacency list and compute in-degrees. Enqueue all courses with in-degree 0. While queue is non-empty, dequeue course, decrement in-degree of prerequisites, and enqueue newly free courses. Valid if count == numCourses.

Time Complexity: O(V + E) - Building graph and processing nodes. Space Complexity: O(V + E) - Adjacency list and queue.

Edge Cases: Self-dependency [0,0], disconnected components, cycle of length 2.

Optimal Python Solution:

```
from collections import deque

def canFinish(numCourses: int, prerequisites: list[list[int]]) -> bool:
    adj = [[] for _ in range(numCourses)]
    indegree = [0] * numCourses
    for crs, pre in prerequisites:
        adj[pre].append(crs)
        indegree[crs] += 1
    q = deque([i for i in range(numCourses) if indegree[i] == 0])
    visited = 0
    while q:
        node = q.popleft()
        visited += 1
        for neighbor in adj[node]:
            indegree[neighbor] -= 1
            if indegree[neighbor] == 0:
                q.append(neighbor)
    return visited == numCourses
```

#127. Word Ladder

HARD

Category: Graphs | Frequently Asked At: Amazon, Google, LinkedIn, Microsoft

Problem Statement:

A transformation sequence from word `beginWord` to word `endWord` using a dictionary `wordList` is a sequence of words beginWord -> s1 -> s2 -> ... -> sk such that every adjacent pair differs by a single letter. Return the number of words in the shortest transformation sequence, or 0 if no such sequence exists.

Constra

Optimal Approach: Bidirectional BFS / Pattern Wildcard Matching

Generate intermediate word patterns (e.g. h*t for hot, hit). Use BFS starting from beginWord to find shortest path to endWord.

Time Complexity: O(M^2 * N) - Where M is word length and N is word count. **Space Complexity: O(M^2 * N) - Adjacency patterns dictionary.**

Edge Cases: endWord not in wordList (returns 0), beginWord equals endWord.

Optimal Python Solution:

```
from collections import deque, defaultdict

def ladderLength(beginWord: str, endWord: str, wordList: list[str]) -> int:
    if endWord not in wordList: return 0
    patterns = defaultdict(list)
    for word in wordList:
        for j in range(len(word)):
            p = word[:j] + "*" + word[j+1:]
            patterns[p].append(word)
    visited = {beginWord}
    q = deque([(beginWord, 1)])
    while q:
        word, length = q.popleft()
        if word == endWord:
            return length
        for j in range(len(word)):
            p = word[:j] + "*" + word[j+1:]
            for neighbor in patterns[p]:
                if neighbor not in visited:
                    visited.add(neighbor)
                    q.append((neighbor, length + 1))
            patterns[p] = []
    return 0
```

#070. Climbing Stairs

EASY

Category: Dynamic Programming | Frequently Asked At: Amazon, Apple, TCS Digital, Infosys DSE

Problem Statement:

You are climbing a staircase. It takes `n` steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Constra

Optimal Approach: Fibonacci DP (Bottom-Up Constant Space)

To reach step n, one can jump from step n-1 or n-2. Thus dp[n] = dp[n-1] + dp[n-2] with base cases dp[1]=1, dp[2]=2.

Time Complexity: O(N) - Linear iteration.

Space Complexity: O(1) - Two variables for previous two steps.

Edge Cases: n = 1 (1), n = 2 (2).

Optimal Python Solution:

```
def climbStairs(n: int) -> int:
    if n <= 2: return n
    one, two = 1, 2
    for _ in range(3, n + 1):
        one, two = two, one + two
    return two
```

#198. House Robber

MEDIUM

Category: Dynamic Programming | Frequently Asked At: Amazon, Google, Microsoft, Cisco

Problem Statement:

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed. Adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night. Given an integer array `nums` representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Constra

Optimal Approach: Dynamic Programming (Rob or Skip)

At house i, max loot is max(loot_without_robbing_current, loot_including_current + loot_two_houses_ago). Maintain two variables rob1 and rob2.

Time Complexity: O(N) - Single pass through houses.

Space Complexity: O(1) - Constant variables.

Edge Cases: 1 house only, 2 houses, zero money houses.

Optimal Python Solution:

```
def rob(nums: list[int]) -> int:
    rob1, rob2 = 0, 0
    for n in nums:
        temp = max(n + rob1, rob2)
        rob1 = rob2
        rob2 = temp
    return rob2
```

#322. Coin Change

MEDIUM

Category: Dynamic Programming | Frequently Asked At: Amazon, Microsoft, Walmart, Flipkart

Problem Statement:

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money. Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

Constra

Optimal Approach: Bottom-Up Dynamic Programming (Unbounded Knapsack)

Let dp[a] be minimum coins for amount a. Initialize dp with inf, dp[0] = 0. For each amount from 1 to amount, dp[a] = min(dp[a], 1 + dp[a - coin]).

Time Complexity: O(amount * len(coins)) - Double loop.

Space Complexity: O(amount) - DP array of size amount + 1.

Edge Cases: amount = 0 (returns 0), impossible amount (returns -1).

Optimal Python Solution:

```
def coinChange(coins: list[int], amount: int) -> int:
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for c in coins:
            if a - c >= 0:
                dp[a] = min(dp[a], 1 + dp[a - c])
    return dp[amount] if dp[amount] != float('inf') else -1
```

#300. Longest Increasing Subsequence

MEDIUM

Category: Dynamic Programming | Frequently Asked At: Google, Amazon, Microsoft, Meta

Problem Statement:

Given an integer array `nums`, return the length of the longest strictly increasing subsequence. Can you solve it in O(n log n) time using patience sorting / binary search?

Constra

Optimal Approach: Binary Search with Patience Sorting (Piles)

Maintain tails array where tails[i] is smallest tail of all increasing subsequences of length i+1. For each x in nums, binary search insertion point into tails.

Time Complexity: O(N log N) - Binary search for each of N numbersSpace Complexity: O(N) - Storage for tails array.

Edge Cases: Strictly decreasing array (1), strictly increasing array (N), duplicates.

Optimal Python Solution:

```
import bisect

def lengthOfLIS(nums: list[int]) -> int:
    tails = []
    for x in nums:
        idx = bisect.bisect_left(tails, x)
        if idx == len(tails):
            tails.append(x)
        else:
            tails[idx] = x
    return len(tails)
```

#1143. Longest Common Subsequence

MEDIUM

Category: Dynamic Programming | Frequently Asked At: Amazon, Microsoft, DoorDash

Problem Statement:

Given two strings `text1` and `text2`, return the length of their longest common subsequence. If there is no common subsequence, return 0.

Constra

Optimal Approach: 2D Dynamic Programming Table

If text1[i] == text2[j], dp[i][j] = 1 + dp[i+1][j+1]. Otherwise, dp[i][j] = max(dp[i+1][j], dp[i][j+1]). Compute bottom-up.

Time Complexity: O(M * N) - Where M, N are string lengths. **Space Complexity: O(min(M, N)) - Space optimized with single row.**

Edge Cases: No common characters (0), identical strings, single character strings.

Optimal Python Solution:

```
def longestCommonSubsequence(text1: str, text2: str) -> int:
    dp = [[0] * (len(text2) + 1) for _ in range(len(text1) + 1)]
    for i in range(len(text1) - 1, -1, -1):
        for j in range(len(text2) - 1, -1, -1):
            if text1[i] == text2[j]:
                dp[i][j] = 1 + dp[i + 1][j + 1]
            else:
                dp[i][j] = max(dp[i + 1][j], dp[i][j + 1])
    return dp[0][0]
```

#072. Edit Distance

MEDIUM

Category: Dynamic Programming | Frequently Asked At: Google, Amazon, Microsoft

Problem Statement:

Given two strings `word1` and `word2`, return the minimum number of operations required to convert `word1` to `word2`. You have three operations permitted on a word: Insert a character, Delete a character, Replace a character.

Constra

Optimal Approach: 2D Dynamic Programming (Insert, Delete, Replace)

If word1[i] == word2[j], cost is dp[i+1][j+1]. Else, cost is 1 + min(insert: dp[i][j+1], delete: dp[i+1][j], replace: dp[i+1][j+1]).

Time Complexity: O(M * N) - Matrix dimensions. **Space Complexity: O(M * N) - 2D DP matrix.**

Edge Cases: One string empty (cost = length of other string), identical words.

Optimal Python Solution:

```
def minDistance(word1: str, word2: str) -> int:
    m, n = len(word1), len(word2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1): dp[i][0] = i
    for j in range(n + 1): dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if word1[i - 1] == word2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
    return dp[m][n]
```

#053. Maximum Subarray

MEDIUM

Category: Greedy | Frequently Asked At: Amazon, Microsoft, Apple, TCS Digital

Problem Statement:

Given an integer array `nums`, find the subarray with the largest sum, and return its sum (Kadane's Algorithm).

Constra

Optimal Approach: Kadane's Algorithm

Track current_sum. At each element, current_sum = max(n, current_sum + n). Update max_sum. Discard negative running sums immediately.

Time Complexity: O(N) - Single pass. **Space Complexity: O(1) - Two variables.**

Edge Cases: All negative numbers (returns largest negative number), single element.

Optimal Python Solution:

```
def maxSubArray(nums: list[int]) -> int:
    max_sum = nums[0]
    curr_sum = 0
    for n in nums:
        curr_sum = max(n, curr_sum + n)
        max_sum = max(max_sum, curr_sum)
    return max_sum
```

#055. Jump Game

MEDIUM

Category: Greedy | Frequently Asked At: Amazon, Microsoft, Meta

Problem Statement:

You are given an integer array `nums`. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position. Return true if you can reach the last index, or false otherwise.

Constra

Optimal Approach: Greedy (Max Reachable Index)

Maintain max_reach = 0. For each index i, if i > max_reach, destination cannot be reached. Update max_reach = max(max_reach, i + nums[i]). If max_reach >= last index, return True.

Time Complexity: O(N) - Single pass. **Space Complexity: O(1) - Constant space.**

Edge Cases: Array with 1 element (always True), zeros blocking further jumps.

Optimal Python Solution:

```
def canJump(nums: list[int]) -> bool:
    max_reach = 0
    for i, n in enumerate(nums):
        if i > max_reach:
            return False
        max_reach = max(max_reach, i + n)
        if max_reach >= len(nums) - 1:
            return True
    return True
```

#056. Merge Intervals

MEDIUM

Category: Intervals | Frequently Asked At: Meta, Amazon, Google, Microsoft

Problem Statement:

Given an array of `intervals` where `intervals[i] = [starti, endi]`, merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Constra

Optimal Approach: Sort by Start Time and Merge Overlaps

Sort intervals by start time. Iterate through intervals: if current interval overlaps with previous (start <= prev.end), merge by setting prev.end = max(prev.end, current.end).

Time Complexity: O(N log N) - Sorting intervals. **Space Complexity: O(N) - Output list.**

Edge Cases: No overlapping intervals, all intervals overlap into one, single interval.

Optimal Python Solution:

```
def merge(intervals: list[list[int]]) -> list[list[int]]:
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

#435. Non-overlapping Intervals

MEDIUM

Category: Intervals | Frequently Asked At: Meta, Amazon, Google

Problem Statement:

Given an array of intervals `intervals` where `intervals[i] = [starti, endi]`, return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Constra

Optimal Approach: Greedy by Earliest End Time

Sort intervals by end time. Keep track of previous end time. If next interval starts before previous end time, an overlap occurs (increment count); otherwise update end time.

Time Complexity: O(N log N) - Sorting by interval end time. **Space Complexity: O(1) - Constant tracking variable.**

Edge Cases: No overlapping intervals (0 removed), all mutually overlapping.

Optimal Python Solution:

```
def eraseOverlapIntervals(intervals: list[list[int]]) -> int:
    intervals.sort(key=lambda x: x[1])
    count = 0
    prev_end = float('-inf')
    for start, end in intervals:
        if start >= prev_end:
            prev_end = end
        else:
            count += 1
    return count
```

#136. Single Number

EASY

Category: Bit Manipulation | Frequently Asked At: Amazon, Google, Apple, TCS Digital

Problem Statement:

Given a non-empty array of integers `nums`, every element appears twice except for one. Find that single one. You must implement a solution with a linear runtime complexity and use only constant extra space (XOR).

Constra

Optimal Approach: Bitwise XOR (a ^ a = 0 and a ^ 0 = a)

XORing any number with itself cancels to 0. XORing all numbers in array leaves only the unique number that appears once.

Time Complexity: O(N) - Single pass XORing all numbers.

Space Complexity: O(1) - Single accumulator variable.

Edge Cases: Single element array, negative numbers.

Optimal Python Solution:

```
def singleNumber(nums: list[int]) -> int:
    res = 0
    for n in nums:
        res ^= n
    return res
```

#191. Number of 1 Bits

EASY

Category: Bit Manipulation | Frequently Asked At: Microsoft, Apple, Amazon, Infosys DSE

Problem Statement:

Write a function that takes the binary representation of a positive integer and returns the number of set bits it has (also known as the Hamming weight).

Constra

Optimal Approach: Brian Kernighan's Algorithm (n & (n - 1))

The operation n & (n - 1) clears the lowest set bit in n. Repeat until n becomes 0; the number of iterations equals the count of 1 bits.

Time Complexity: O(K) - Where K is number of set bits (at most 32 for 32-bit integer). **Space Complexity: O(1) - Constant space.**

Edge Cases: n = 0 (0 bits), n = 2^31 - 1 (all ones).

Optimal Python Solution:

```
def hammingWeight(n: int) -> int:
    count = 0
    while n:
        n &= (n - 1)
        count += 1
    return count
```

#338. Counting Bits

EASY

Category: Bit Manipulation | Frequently Asked At: Amazon, Google, Apple

Problem Statement:

Given an integer `n`, return an array `ans` of length `n + 1` such that for each `i` (0 <= i <= n), `ans[i]` is the number of 1's in the binary representation of i in O(n) time.

Constra

Optimal Approach: Dynamic Programming with Bit Shifting (dp[i] = dp[i >> 1] + (i & 1))

Count of set bits in i equals bits in (i // 2) plus the least significant bit (i % 2). Compute dp array iteratively from 0 to n.

Time Complexity: O(N) - Single linear pass to fill array.

Space Complexity: O(N) - Output array of size n + 1.

Edge Cases: n = 0 (returns [0]), powers of 2 (always 1 bit).

Optimal Python Solution:

```
def countBits(n: int) -> list[int]:
    dp = [0] * (n + 1)
    for i in range(1, n + 1):
        dp[i] = dp[i >> 1] + (i & 1)
    return dp
```

#371. Sum of Two Integers

MEDIUM

Category: Bit Manipulation | Frequently Asked At: Meta, Google, Microsoft

Problem Statement:

Given two integers `a` and `b`, return the sum of the two integers without using the operators + and - (using bitwise XOR and AND).

Constra

Optimal Approach: Bitwise Addition (XOR for Sum, AND with Shift for Carry)

a ^ b gives sum without carry. (a & b) << 1 gives the carry bits. Repeat addition with sum and carry until carry is 0 (with 32-bit mask for 2's complement).

Time Complexity: O(1) - Maximum 32 bit operations. **Space Complexity: O(1) - Constant variables.**

Edge Cases: Negative numbers, opposite sign numbers, sum resulting in 0.

Optimal Python Solution:

```
def getSum(a: int, b: int) -> int:
    mask = 0xFFFFFFFF
    while (b & mask) > 0:
        carry = (a & b) << 1
        a = a ^ b
        b = carry
    return (a & mask) if b > 0 else a
```